

**Filière : 4IASDR G : 8**

**Projet : Chatbot RAG sur CV avec Streamlit**

---

# **COMPTE RENDU TECHNIQUE**

**Conception d'un Chatbot RAG sur CV**

---

**Réalisé par :**  
Abdellah Elouali

**Année Universitaire 2025-2026**

# Sommaire

|          |                                                             |          |
|----------|-------------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                         | <b>2</b> |
| <b>2</b> | <b>Architecture Globale du Système</b>                      | <b>2</b> |
| <b>3</b> | <b>Justification des Choix Technologiques</b>               | <b>2</b> |
| 3.1      | Interface Utilisateur : Streamlit . . . . .                 | 2        |
| 3.2      | Traitement Documentaire et Chunking : LangChain . . . . .   | 2        |
| 3.3      | Modèle d'Embedding : BAAI/bge-m3 . . . . .                  | 3        |
| 3.4      | Base de Données Vectorielle : ChromaDB . . . . .            | 3        |
| 3.5      | LLM (Génération) : Llama-3.3-70b . . . . .                  | 3        |
| <b>4</b> | <b>Implémentation des Fonctionnalités Clés</b>              | <b>3</b> |
| 4.1      | Chargement Dynamique et Gestion de la Mémoire . . . . .     | 3        |
| 4.2      | Stratégie de Prompt Engineering (Sécurité Métier) . . . . . | 3        |
| 4.3      | Aperçu de l'Interface Finale . . . . .                      | 4        |
| <b>5</b> | <b>Conclusion</b>                                           | <b>4</b> |

# 1. Introduction

Ce document détaille les choix de conception et d'architecture réalisés dans le cadre du développement d'un chatbot basé sur l'approche RAG (*Retrieval-Augmented Generation*). L'objectif principal de cette application est de permettre à un utilisateur de dialoguer avec un système expert capable de répondre à des questions de manière stricte, en se basant exclusivement sur le contenu d'un Curriculum Vitae (CV) fourni dynamiquement via une interface web.

## 2. Architecture Globale du Système

L'application repose sur un pipeline modulaire divisé en deux grandes phases :

- **Phase d'Indexation (Ingestion)** : Chargement du document PDF brut, nettoyage, découpage en segments textuels (*chunking*), vectorisation (*embedding*) et stockage dans une base de données vectorielle.
- **Phase de Restitution (Génération)** : Interrogation par l'utilisateur, recherche de similarité sémantique pour retrouver les fragments pertinents, construction d'un prompt contraint et génération de la réponse par un *Large Language Model* (LLM).

## 3. Justification des Choix Technologiques

Afin de répondre aux exigences de performance, de modularité et de respect du périmètre fonctionnel, les technologies suivantes ont été sélectionnées :

### 3.1. Interface Utilisateur : Streamlit

- ▷ **Choix** : Le framework Streamlit a été privilégié pour le développement du front-end.
- ▷ **Justification** : Streamlit permet de concevoir rapidement des interfaces web interactives en pur Python. Il intègre nativement des composants de gestion d'état (`st.session_state`) et des interfaces conversationnelles (`st.chat_message`, `st.chat_input`), idéales pour construire un chatbot fonctionnel sans nécessiter de développement JavaScript ou CSS complexe.

### 3.2. Traitement Documentaire et Chunking : LangChain

- ▷ **Choix** : Utilisation de PyPDFLoader et RecursiveCharacterTextSplitter issus de la bibliothèque LangChain.
- ▷ **Justification** : PyPDFLoader extrait efficacement le texte brut des CV tout en conservant les métadonnées de la source (indispensable pour tracer la provenance des réponses). Le RecursiveCharacterTextSplitter a été configuré avec une taille de chunk de 500 caractères et un chevauchement (*overlap*) de 50 caractères (soit 10 %). Cette méthode de découpe récursive évite de tronquer une phrase ou une idée en plein milieu, préservant ainsi le contexte sémantique crucial pour le modèle d'*embedding*.

### 3.3. Modèle d'Embedding : BAAI/bge-m3

- ▷ **Choix** : Modèle BAAI/bge-m3 (implémenté via la bibliothèque `sentence-transformers`).
- ▷ **Justification** : Ce modèle génère des vecteurs denses de 1024 dimensions avec une grande précision d'alignement. Son choix est stratégique car la mention "M3" désigne ses fortes capacités multilingues. Un CV contemporain contient très souvent un mélange de français et de terminologies techniques anglophones (ex : *Data Engineering*, *Machine Learning*). Ce modèle excelle particulièrement dans l'alignement sémantique inter-langues.

### 3.4. Base de Données Vectorielle : ChromaDB

- ▷ **Choix** : ChromaDB en mode local (`PersistentClient`).
- ▷ **Justification** : ChromaDB est une base de données vectorielle open-source, légère, fonctionnant directement en Python et très optimisée pour ce type de cas d'usage. Le choix du `PersistentClient` assure la persistance de l'indexation sur le disque. Le calcul de similarité a été explicitement configuré sur l'espace cosinus (`hnsw:space: cosine`), métrique de référence pour comparer la similarité angulaire des `embeddings` textuels denses.

### 3.5. LLM (Génération) : Llama-3.3-70b

- ▷ **Choix** : Utilisation du modèle `llama-3.3-70b-versatile` exécuté via l'API ultra-rapide Groq (utilisant un client compatible OpenAI).
- ▷ **Justification** : L'exigence stricte de ce projet réside dans le refus absolu de répondre aux questions hors-périmètre (anti-hallucination). Il est prouvé que les grands modèles de langage, comme Llama-3 à 70 milliards de paramètres, respectent beaucoup mieux les instructions négatives complexes ("Ne réponds pas si...") que les modèles plus restreints. La température a été fixée très bas ( $T = 0.1$ ) pour forcer l'IA à adopter un comportement purement factuel, déterministe et ancré dans le contexte fourni.

## 4. Implémentation des Fonctionnalités Clés

### 4.1. Chargement Dynamique et Gestion de la Mémoire

L'une des contraintes majeures de l'application était de permettre un rechargement complet de la base à chaque nouvel ajout de CV. Puisque Streamlit charge par défaut les fichiers uploadés directement en mémoire vive (RAM), un pont technique a été réalisé à l'aide de la bibliothèque native `tempfile` de Python. Cette technique permet de matérialiser un fichier PDF physique temporaire sur le disque — une condition indispensable pour que l'extracteur `PyPDFLoader` puisse fonctionner correctement — avant d'être supprimé automatiquement pour prévenir toute fuite de mémoire.

### 4.2. Stratégie de Prompt Engineering (Sécurité Métier)

Pour garantir la sécurité métier et interdire à l'IA d'utiliser ses connaissances pré-entraînées sur Internet, une approche de *Zero-Shot Prompting* très stricte a été implé-

mentée. Le prompt envoyé au LLM est systématiquement architecturé en trois blocs distincts :

1. **Persona & Règles Métier** : Définition intransigeante du rôle et des limites ("*Tu es un assistant RAG strict... N'invente aucune information*").
2. **Contexte Dynamique (CONTEXT)** : Injection de la concaténation des textes extraits par les chunks retournés par ChromaDB (Top-K = 3).
3. **Requête Utilisateur (QUESTION)** : La question brute posée par l'utilisateur.

Le système exige également un format de sortie standardisé découpé en trois champs (Réponse, Contexte utilisé, Limite). Si la réponse pertinente ne se trouve pas dans les fragments récupérés, la consigne ordonne explicitement au LLM de déclencher la phrase de repli unique : "*Je ne trouve pas la réponse dans les documents fournis*".

### 4.3. Aperçu de l'Interface Finale

L'image ci-dessous présente l'interface utilisateur développée avec Streamlit. On y retrouve la zone de téléchargement du CV à gauche, et l'espace de dialogue interactif au centre.

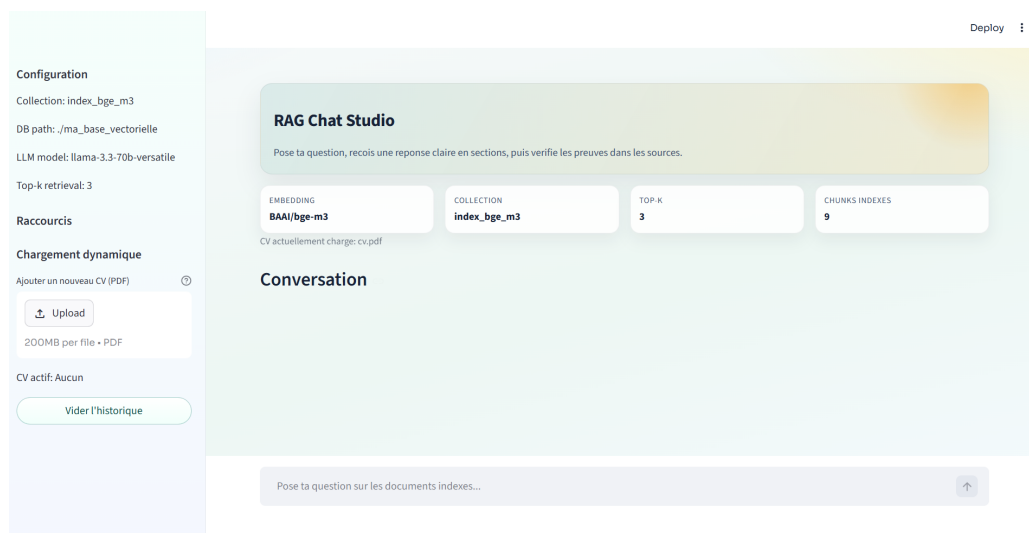


FIGURE 1 – Interface de l'application Chatbot RAG sur Streamlit

## 5. Conclusion

Le pipeline final mis en place répond en tout point au cahier des charges exigé. L'encapsulation de chaque logique métier dans des fonctions dédiées (chargement des variables via `dotenv`, chunking, embedding, recherche sémantique, post-traitement des réponses du LLM) rend le code robuste et maintenable. L'application Streamlit développée offre une interface utilisateur élégante et réactive, capable de gérer dynamiquement de nouveaux CVs tout en garantissant un très haut niveau de fiabilité factuelle grâce au puissant tandem **ChromaDB + BGE-M3 + Llama-3**.